

Lab #11

In this lab, you will complete a C program that manages software package dependencies using a linked list. The program reads package information from a text file, builds a linked list of packages with their dependency relationships, and then checks whether developers can install their desired packages based on what they already have installed.

A file named `packages.txt` holds information about software packages and their dependencies. Each line defines a package. The first word is the package name. The remaining words on the same line are the names of its dependency packages. A package cannot be installed unless all of its dependencies are already installed. Dependencies are always defined before the packages that depend on them.

Example file content (`packages.txt`):

```
gcc
make gcc
cmake make
valgrind gcc make
```

In this example, there are 4 packages. `gcc` has no dependencies. `make` depends on `gcc`. `cmake` depends on `make`. `valgrind` depends on both `gcc` and `make`. Assume that there will not be more than 10 dependencies for any package.

A C source file is provided to you. It contains the general program structure, the main function, helper functions, and missing parts marked with TODO comments. Your task is to complete these missing parts.

Required Data Structures

The following structs are already defined in the `Lab11.c` file. Do not modify them.

package_t — Represents a software package:

```
typedef struct package {
    char name[MAX_NAME_LEN];           /* package name */
    struct package* dependencies[MAX_DEPS]; /* dependency pointers */
    int dep_count;                      /* number of dependencies */
} package_t;
```

node_t — Linked-list node for storing packages:

```
typedef struct node {
    package_t* data; /* points to a dynamically created package_t */
    struct node* next;
} node_t;
```

developer_t — Represents a developer:

```
typedef struct developer {
    char name[30];           /* developer name */
    package_t* installed[MAX_DEPS]; /* installed packages */
    package_t* to_install[MAX_DEPS]; /* packages to install */
    int inst_count;          /* installed count */
    int want_count;          /* to-install count */
} developer_t;
```

Provided Helper Functions

The following functions are implemented in the `Lab11.c` file. You do not need to implement them, but you should use them in your solution.

```
int getnextline(FILE* fp, char* buffer);
/* Reads the next line from fp into buffer.
   Returns 1 on success, 0 at end of file. */

void splitline(char* line, char parts[][MAX_NAME_LEN],
               int* number_of_parts);
/* Splits a line into space-separated words.
   parts[0], parts[1], ... hold the words.
   number_of_parts holds the total count. */
```

Functions to Implement

You must implement the following two functions in the provided C file. You may also write additional helper functions (e.g., a function to find a package by name in the linked list).

Function 1: `node_t* read_packages(char* filename)`

This function reads the file specified by `filename`. Each line defines a package: the first word is the package name, and the remaining words are the names of its dependencies. The function must:

- Open the file.
- Read each line using `getline`.
- Split each line using `splitline`.
- Dynamically allocate a `package_t` for each package.
- Store the package name and find its dependency packages in the already-built linked list.
- Dynamically allocate a `node_t` and append it to the linked list.
- Return the head pointer of the linked list.

Hint: Since dependencies are always defined before the packages that depend on them, you can search the linked list built so far to find each dependency by name.

Function 2: `void check_installations(developer_t* developers, int num_developers, node_t* package_list)`

For each developer, this function checks every package in their `to_install` list. For each such package, it verifies that **all** of its dependencies are present in the developer's `installed` list. If any package cannot be installed (i.e., a dependency is missing), the function prints the developer's name.

Sample Run

Given the `packages.txt` file shown above, with Alice having `gcc` and `make` installed (wanting `cmake`), and Bob having only `gcc` installed (wanting `cmake` and `valgrind`):

```
Loaded packages:
gcc
make (depends on: gcc)
cmake (depends on: make)
valgrind (depends on: gcc make)

Developers who cannot install all desired packages:
Bob
```

Alice can install `cmake` because its dependency `make` is already installed. Bob cannot install `cmake` because he is missing `make`, so his name is printed.

Important Notes

- Do not change the given main function.
- Complete only the missing parts marked with `TODO`.
- Use `malloc` to dynamically allocate `package_t` and `node_t` instances.
- Use pointer comparison (not string comparison) to check if a dependency is in the installed list, since both point to the same `package_t` instance.
- Make sure your program compiles and runs correctly with the provided `packages.txt` file.
- You may write additional helper functions as needed.